LZSCC.311 Software Design Studio Project II (Networked Studio)

Assessment Description

1 Introduction

LZSCC.311 provides students with the opportunity to build a networked software system using modern architectural patterns and agile development practices. The focus is on building a real-world application using microservices architecture, event-driven systems, and continuous integration (CI) practices.

In this setting, you are expected to work continuously and self-driven on the given software development task and cooperate and communicate frequently with your group members. The development process adopts Scrum methodology with emphasis on disciplined software engineering practice, including sprint planning, daily standups, and retrospectives.

Generative AI Policy: AMBER

AMBER CATEGORY
Under this category, Gen AI tools may be used in an assistive role, as specified by the module convenor.

GenAI Policy: You **may** use generative AI tools in an **assistive role** only, i.e., chat assistants (e.g., ChatGPT, Claude) for explanations, brainstorming, and troubleshooting, and inline code completion (e.g., GitHub Copilot autocomplete). The following is **not permitted**:

- **Agentic AI tools:** tools that autonomously plan, write, run, or modify code across your repository (e.g., Claude Code, OpenAI Codex, Cursor Agent, GitHub Copilot agent mode or coding agent, Devin).
- **Outsourcing the task:** entering this assessment description, or large parts of it, into an AI tool to generate the system, its documentation, the presentation slides, or the reflective report.

You must **declare your GenAI usage** in your reflective report (see Section 3.5). You are expected to understand and be able to explain all code you submit, regardless of how it was generated; this is checked in the presentations (see Section 5.1). Use of non-permitted tools may be treated as academic misconduct under university regulations.

2 Task

This year's project is:

2.1 MARP-Guide: RAG Chatbot with Microservices and Event-Driven Architecture

2.1.1 Product Goal

Build a chat application that answers questions about Lancaster University's **Manual of Academic Regulations and Procedures (MARP)**. Answers must be derived from the processed MARP PDF documents, properly cited (with title, page number, and link), and presented in an understandable manner.

Target Audience: Students and staff who need quick access to university regulations.

Value: Reliable, quickly accessible information with sources—no hallucinations.

2.1.2 Framework and Constraints

You are asked to work on this project in groups of 3–4 persons.

- **Architecture:** Microservice architecture **and** Event-Driven Architecture (EDA). Implement decoupled services for ingestion, indexing/retrieval, and chat orchestration.
- **RAG (Retrieval-Augmented Generation):** Mandatory. Use hybrid or dense retrieval, but answers must **only** contain information supported by sources.
- **LLM:** Use **OpenRouter** `https://openrouter.ai/` (free models are permitted for this course).
  - **No sensitive data** may be sent to external APIs. MARP documents are public, so this is acceptable.
- **Data Source:** Main page: `https://www.lancaster.ac.uk/academic-standards-and-quality/regulations-and-policies/manual-of-academic-regulations-and-procedures/`
  - Linked PDF documents form the knowledge base of your system.
- **Team Size:** 3–4 persons
- **Duration:** 10 weeks (300 hours per person: 30 contact hours and 270 hours of independent learning)
- **Methodology:** Scrum (short sprints, reviews, retrospectives, visible artefacts)

2.1.3 Minimum Functionality (MVP)

A. **Ingestion:** Automatically discover and fetch MARP PDFs, storing text and metadata (title, URL, date, page numbers).
B. **Indexing:** Implement chunking and embeddings (choice of model is yours), preserve metadata.
C. **Retrieval:** Return top-k snippets reproducibly via API.
D. **RAG Answers:** Provide chat endpoint/UI that generates answers with **at least 2 citations** (title, page, link). *Note: Assessment 1 (first increment) requires at least 1 citation; final MVP requires at least 2 citations.*
E. **EDA Flow:** Key workflow steps must generate events (e.g., `DocumentDiscovered`, `DocumentExtracted`, `ChunksIndexed`, `AnswerGenerated`).
F. **Operations:** Local deployment via **Docker Compose**. Health checks for each service.
G. **Quality:** Logs for all services.

2.1.4 Technology Constraints and Implementation Requirements

To ensure adequate complexity and learning depth, the following constraints apply. If you are unsure, please ask the module convenor.

Prohibited Technologies:

The following frameworks and pre-built solutions, as well as functionally similar ones, are **not permitted**:

- **RAG Frameworks:** OpenWebUI, anything-llm, Haystack, LlamaIndex, ...
- **AI Agent Frameworks:** Langchain, LangGraph, Pydantic AI, CrewAI, AutoGen, ...
- **Pre-built Chat Applications:** Complete chat application templates or RAG tutorial projects that provide more than 50% of required functionality

Permitted Technologies:

The following frameworks and pre-built solutions, as well as functionally similar ones, are **permitted**:

- **HTTP/API Clients:** Standard libraries (requests, httpx, fetch) for direct OpenRouter API calls

- **UI Frameworks:** React, Vue, Svelte, Angular, and UI component libraries (Material-UI, Chakra UI, shadcn/ui)
- **Backend Frameworks:** FastAPI, Flask, Express.js, NestJS, and database ORMs (SQLAlchemy, Prisma, TypeORM)
- **Infrastructure:** Messaging systems (RabbitMQ, Kafka, Redis Pub/Sub), vector databases (Qdrant, Weaviate, ChromaDB, pgvector), SQL/NoSQL databases, Docker and Docker Compose
- **ML/NLP Libraries:** sentence-transformers (for embeddings), Hugging Face transformers, OpenAI Python SDK (only for OpenRouter-compatible calls), tiktoken (token counting), PyPDF2, pdfplumber (PDF parsing)

Core Implementation Requirements:

The following components **must be implemented from scratch** by your team:

A. **RAG Pipeline Core:**
   - Custom chunking strategy with justification
   - Custom retrieval orchestration logic (query → embedding → search → ranking)
   - Custom citation extraction and formatting logic
   - Custom prompt templates for RAG responses

B. **Event-Driven Architecture:**
   - At least 5 custom event types defined with schemas
   - Custom event handler implementations
   - Documented event flow through the system

C. **Microservices Architecture:**
   - At least 3 independent services with clear responsibilities
   - Custom-designed REST/GraphQL APIs with documentation
   - Inter-service communication (both synchronous and event-based)

Required Additional Features:

Each team must implement **exactly 2 additional features** beyond the MVP:

- **1× Feature from Tier 1 (Foundation Features)** — extends basic infrastructure
- **1× Feature from Tier 2 (Advanced Features)** — adds significant technical complexity

Teams select their features during Sprint Planning (Week 1) and must document the design in Assessment 1 and demonstrate implementation in Assessment 2.

Tier 1: Foundation Features

A. **User Authentication & Session Management:** Login/logout, session handling, user-specific chat isolation
B. **Chat History Persistence:** Store all chats in database, retrieve past conversations
C. **Bookmarks & Favourites:** Users can mark important messages, retrieve saved items
D. **Document Management Interface:** Admin interface for manual PDF upload, document status overview
E. **Basic Monitoring Dashboard:** Service health checks, indexing statistics, request counts

Tier 2: Advanced Features

A. **Hybrid Search:** Combine dense vector search with sparse retrieval (BM25), weighted fusion
B. **Query Rewriting & Expansion:** Automatic query reformulation, synonym expansion, LLM-based rewriting
C. **Re-Ranking Pipeline:** Two-stage retrieval (recall → precision) with separate re-ranking model
D. **Multi-Model Comparison:** Parallel answer generation with different LLMs, side-by-side UI comparison

E. **User Feedback & Quality Metrics:** Thumbs up/down, feedback tracking, confidence scoring
F. **Advanced Monitoring & Observability:** Distributed tracing, latency metrics per service, event flow visualisation
G. **Rate Limiting & Intelligent Caching:** Request limits per user/service, semantic caching for similar queries
H. **A/B Testing Infrastructure:** Test different retrieval strategies in parallel, automated metric collection

Networked System Design:

The application must demonstrate proper networked system architecture with multiple services communicating over network protocols. All inter-service communication must be explicitly designed and documented. No in-process communication between services is allowed—all services must communicate via network interfaces (HTTP REST APIs or message queues).

2.1.5 Non-Goals

- No personal advice or decision recommendations; **citations and links only**.
- No user login required **(unless chosen as Tier 1 additional feature)**.
- No completeness guarantee for all university documents—focus is the **MARP area**.
- No use of pre-built RAG frameworks or AI agent frameworks (see Technology Constraints).

2.1.6 Acceptance Criteria (Definition of Done)

- **Correctness & Sources:** ≥ 90% of answers in the demo catalogue contain **correct** citations (title + page + link).
- **Fallback:** If no suitable source exists → explicitly state "not certain / no source found".
- **Reproducibility:** `docker compose up` brings the system up and running (README with commands).
- **Network Architecture:** All services communicate via network protocols (HTTP REST APIs or message queues); no in-process communication between services allowed.
- **Technology Compliance:** No prohibited frameworks used; core RAG pipeline, EDA, and microservices implemented from scratch as specified.
- **Additional Features:** Both required features (1× Tier 1, 1× Tier 2) documented in Assessment 1, fully implemented in Assessment 2.
- **Scrum Evidence:** Product goal, prioritised backlog, sprint artefacts, reviews/retrospectives documented.
- **OpenRouter Compliance:** No upload of sensitive data; MARP text snippets only in RAG context to LLM.

2.1.7 Example Milestones

Sprint 1 (Weeks 1–5):

- **M1 (Week 1):** Team formation; Scrum fundamentals; Sprint planning; feature selection (Tier 1 + Tier 2); repository setup + GitHub Projects configured
- **M2 (Week 2):** Microservices architecture designed; tech decisions documented; ingestion service started
- **M3 (Week 3):** Ingestion service functional (discover + fetch MARP PDFs); Docker Compose setup; Event-Driven Architecture applied; first events (`DocumentDiscovered`, `DocumentExtracted`)
- **M4 (Week 4):** Indexing service (chunking + embeddings); retrieval service (`/search` endpoint); basic RAG (`/chat` endpoint with 1 citation); 3+ events total

- **M5 (Week 5):** System integration verified; Docker Compose fully functional; Assessment 1 presentation (12.11.2026)

Sprint 2 (Weeks 6–10):

- **M6 (Week 6):** Sprint 1 retrospective completed; Sprint 2 planning finished; additional features work started
- **M7 (Week 7):** Chat UI implemented; complete RAG (2 citations); 5+ events implemented; automated tests started; CI pipeline planned
- **M8 (Week 8):** Additional features (Tier 1 + Tier 2) functional; automated tests implemented; GitHub Actions CI pipeline operational
- **M9 (Week 9):** Test catalogue (10–15 questions) completed; acceptance criteria verified; final integration and bug fixes
- **M10 (Week 10):** Final system polish and documentation; Assessment 2 presentation (17.12.2026)

2.1.8 Suggested Initial Backlog

- **EPIC Ingestion:** Discover MARP links → store PDFs → `DocumentDiscovered`
- **EPIC Extraction:** PDF→Text+Metadata → `DocumentExtracted`
- **EPIC Index:** Chunking+Embedding → `ChunksIndexed`
- **EPIC Retrieval:** `/search` returns top-k snippets with pages/URLs
- **EPIC RAG & Chat:** `/chat` generates answer with ≥2 citations, `AnswerGenerated`
- **EPIC UX:** Web chat with sources panel & link-out
- **EPIC Quality:** Measurements, test catalogue, fallbacks

Acceptance (per story): measurable, demo-capable, checkable in DoD checklist.

2.1.9 Data Protection and Ethics

- No personal data or confidential content may be sent to OpenRouter.
- Answers must always include sources; communicate clearly when uncertain.
- Respect robots/policies of university site; process only publicly accessible PDFs.

3 Assessment Structure

The module follows a two-increment Scrum approach with an assessment after each 5-week sprint. Each assessment is worth **50% of your final module grade** and consists of three graded components:

- **Group Presentation** (25% of the assessment): architecture overview, live demonstration, code walkthrough, and questions (marked as described in Section 5.1).
- **Group Code & Scrum Process** (25% of the assessment): assessed after the deadline from the submitted repository snapshot (Section 5.2).
- **Individual Reflective Report** (50% of the assessment): 2 pages maximum (Section 5.3).

3.1 Assessment 1: First Increment — Core RAG Pipeline (50%)

Upload deadline: 11.11.2026, 12:00 (Moodle), the day before the presentations
Presentations: 12.11.2026, during the scheduled studio session (team time slots will be published on Moodle)
Focus: Functional core system with basic RAG capability, microservices architecture, event-driven design, and comprehensive documentation.

Required Functionality:

- **Ingestion Service:** MARP PDFs discovered, fetched, and parsed (text + metadata)

- **Indexing Service:** Chunking, embeddings, and vector database storage functional
- **Retrieval Service:** API endpoint `/search` returns top-k snippets with metadata (title, page, URL)
- **Basic RAG:** Chat endpoint `/chat` generates answers with at least 1 citation
- **Event-Driven Architecture:** Minimum 3 events implemented (e.g., `DocumentDiscovered`, `ChunksIndexed`, `RetrievalCompleted`)
- **Deployment:** `docker compose up` brings system online
- **Network Architecture:** All services communicate via network protocols (HTTP REST or message queue)

Submission Requirements:

Each student submits to Moodle by the upload deadline:

- Repository snapshot: zip of the repository including the `.git` folder (commit history), without build artefacts such as `node_modules/`, `.venv/`, `target/`
- Link to the team's GitHub repository, with access granted to both markers (supplementary evidence)
- Individual reflective report (PDF, 2 pages maximum, plus GenAI declaration appendix)
- Presentation slides (PDF or PowerPoint)

The repository must contain:

- README.md with:
  - High-level architecture overview (Mermaid diagrams recommended)
  - Setup and run instructions (`docker compose up` must work)
  - Technology stack overview
- Complete source code with `docker-compose.yml`
- `/docs` folder containing:
  - Detailed architecture diagrams (microservices + network communication patterns)
  - Event schemas for implemented events with example payloads
  - API specification for REST endpoints
  - Additional features design (documented design and implementation plan for chosen Tier 1 and Tier 2 features)
  - Scrum artefacts (backlog, sprint log, team retrospective)

Additional Features Design (expected content):

For each of the two chosen features (Tier 1 and Tier 2), the `/docs` folder should contain a short design document (approx. 1–2 pages per feature) covering:

A. **What:** the chosen feature, why the team chose it, and 2–3 user stories with acceptance criteria
B. **Where it fits:** which services are new or changed, shown in an updated architecture diagram
C. **Interfaces:** new or changed REST endpoints, new events with draft schemas, and changes to the data model
D. **How:** the technical approach and key design decisions, with at least one alternative considered
E. **Plan:** the work broken down into backlog items (GitHub Issues) for Sprint 2, with rough estimates and owners
F. **Verification:** how the team will test that the feature works

Graded Components:

- **Group Presentation:** 25% of Assessment 1 = 12.5% of overall module grade
- **Group Code & Scrum Process:** 25% of Assessment 1 = 12.5% of overall module grade

- **Individual Reflective Report:** 50% of Assessment 1 = 25% of overall module grade

3.2 Assessment 2: Second Increment — Production-Ready System (50%)

Upload deadline: 16.12.2026, 12:00 (Moodle), the day before the presentations
Presentations: 17.12.2026, during the scheduled studio session (team time slots will be published on Moodle)
Focus: Complete MVP with all acceptance criteria met, both additional features implemented, automated testing, and CI pipeline operational.

Required Functionality:

- **Complete RAG:** Chat generates answers with at least 2 citations (as per MVP)
- **Full Event-Driven Architecture:** Minimum 5 events implemented with complete event flow documentation
- **Chat UI:** Web interface or API-based chat interaction
- **Additional Features Implemented:** Both chosen features (1× Tier 1 + 1× Tier 2) fully functional
- **Automated Tests:** Unit and integration tests (minimum 10–15 test cases), test catalogue with demo questions
- **CI Pipeline:** GitHub Actions running tests on push
- **Acceptance Criteria Met:** ≥90% correct citations in demo catalogue, proper fallback handling

Submission Requirements:
Each student submits to Moodle by the upload deadline:

- Repository snapshot: zip of the repository including the `.git` folder (commit history), without build artefacts such as `node_modules/`, `.venv/`, `target/`
- Link to the team's GitHub repository, with access granted to both markers (supplementary evidence)
- Individual reflective report (PDF, 2 pages maximum, plus GenAI declaration appendix)
- Presentation slides (PDF or PowerPoint)

The repository must contain:

- README.md with:
  - Updated high-level architecture overview (reflecting changes since Assessment 1)
  - Setup and run instructions (`docker compose up` must work)
  - Technology stack overview
- Complete source code with `docker-compose.yml`
- `/docs` folder containing:
  - Updated detailed architecture diagrams (changes since Assessment 1 highlighted)
  - Complete event schemas (5+ events) with event flow documentation
  - Updated API specification for all REST endpoints
  - Additional features documentation (design, implementation, and rationale for Tier 1 + Tier 2 features)
  - Test documentation (coverage report, test catalogue with 10–15 demo questions)
  - Scrum artefacts (updated backlog, sprint log, lessons learnt, final retrospective)
- Complete test suite (unit and integration tests)
- CI configuration (GitHub Actions workflow files)

Graded Components:

- **Group Presentation:** 25% of Assessment 2 = 12.5% of overall module grade
- **Group Code & Scrum Process:** 25% of Assessment 2 = 12.5% of overall module grade
- **Individual Reflective Report:** 50% of Assessment 2 = 25% of overall module grade

3.3 Summary: Graded Components

There are **six graded components** in total:

Component	Assessment 1	Assessment 2	Total
Group Presentation	12.5%	12.5%	25%
Group Code & Scrum Process	12.5%	12.5%	25%
Individual Reflective Report	25%	25%	50%
Total	**50%**	**50%**	**100%**

3.4 Presentation Format

Each team presentation should allocate approximately **5 minutes per group member** (e.g., a team of 4 has 20 minutes presentation time), followed by up to 10 minutes of questions and discussion. Presentations are stopped at the end of the allocated time.

Presentation Structure:

- Brief overview of the **architecture** (services, network communication, event flow) and key design decisions
- **Live demonstration** of the functionality of the increment
- **Walkthrough of selected code**
- Every team member presents part of the presentation and must be able to explain the system and their own contribution

Please start your presentation with the system **already running**. There is no need to show the system being started (`docker compose up`), the test runs, or the Scrum artefacts during the presentation—these are assessed from the submitted repository (see Section 5.2).

Like in real-world software development, the module convenor and marker may interrupt the planned demo and ask questions to direct what is shown.

Marking: The presentation receives a **group mark**, except for the criterion *Technical Understanding*, which is marked **individually** for each student (see Section 5.1).

3.5 Individual Reflective Report Format

Each report should contain **2 pages maximum**, without a cover page (the GenAI declaration appendix does not count towards the page limit).

The report is a **technical reflection** on your individual contribution to the project. It should describe **what you did, why you did it, and what you learned**. The focus is on your thought process, decision-making, and learning, not just listing features.

Required Content:

- **Contribution & Collaboration (approx. 0.5 pages):** What did you implement in this increment (with references to your code and commits)? Who did you work with, and how did you divide responsibilities?

- **Technical Decisions (approx. 0.75 pages):** Which architectural or design decisions did you make or contribute to? Which alternatives did you consider, and why did you choose your approach?
- **Challenges & Learning (approx. 0.75 pages):** What were the main technical, team, or process challenges? How did you address them, and how effective was this? What did you learn, and what would you do differently next time?

How to Reference Your Code:

Throughout your report, reference specific code you wrote using file paths and line numbers. For example:

- "I implemented the chunking algorithm in `src/indexing/chunker.py:45-120`..."
- "The event handler for `DocumentDiscovered` is in `src/events/handlers.py:34`..."

This helps markers connect your reflection to your actual code contributions. However, remember that code quality is assessed from the submitted repository (Code & Scrum Process), not from the report.

GenAI Declaration:

In line with the **AMBER** GenAI policy, add a short **GenAI declaration** as an appendix (max. half a page; not counted towards the page limit and not marked). State:

- Which GenAI tools you used (e.g., ChatGPT, Claude, GitHub Copilot autocomplete)
- For which purposes you used them
- How you checked and adapted the generated content

If you did not use any GenAI tools, state "No GenAI tools used".

Collaborative Work:

When describing contributions you led, state who helped you. When describing contributions you assisted with, state who led. For example:

- "I led the implementation of the retrieval service (`src/retrieval/`), with Alice helping on the ranking algorithm."
- "I assisted Bob with the ingestion pipeline, specifically implementing the PDF metadata extraction."

The report should describe your **individual** contribution, but acknowledge collaboration honestly.

Assessment Schedule

Assessment	Upload Deadline	Presentation	Weight (%)
Assessment 1 (First Increment: Core RAG Pipeline)	11.11.2026, 12:00	12.11.2026	50 (Presentation: 12.5%, Code & Scrum: 12.5%, Report: 25%)
Assessment 2 (Second Increment: Production-Ready System)	16.12.2026, 12:00	17.12.2026	50 (Presentation: 12.5%, Code & Scrum: 12.5%, Report: 25%)

4 Support

Support will be provided throughout the module through the weekly design studio sessions and the office hour (**Tuesdays, 10:00–11:00**, or by appointment) for technical and non-technical questions.

Since this is a software development project in the third year, you are expected to work **self-driven** and **ask questions when necessary**. As is normal with projects, small delays can happen, so start work early and plan ahead.

5 Marking

Each of the three components below is marked out of **40 marks**, and the same marking scheme applies to Assessment 1 and Assessment 2. The marks are converted to percentages and weighted as shown in Section 3.3. For every criterion, the marker records the mark with a short justification on a marking sheet, which is used for moderation with the second marker and for your feedback.

5.1 Presentation Marking (40 marks)

Worth **25% of each assessment** (12.5% of the final module grade). The criteria are deliberately broad so that they can be assessed reliably within the presentation time.

Criterion	What is assessed	Marks
Architecture & Design [Group]	Architecture, service communication, and event flow are explained clearly with appropriate diagrams; key decisions are justified.	12
Demonstrated Functionality [Group]	The required functionality of the increment is shown working live.	12
Code Walkthrough [Group]	Selected code is shown and explained, and it matches the presented architecture.	8
Technical Understanding [Individual]	The student answers questions about their own contribution and the system correctly.	8
Total		**40**

Each student's presentation mark consists of the team's group marks (32) plus their individual *Technical Understanding* mark (8).

5.2 Code & Scrum Process Marking (40 marks)

Worth **25% of each assessment** (12.5% of the final module grade). This group component is assessed **after the deadline** from the submitted repository snapshot, including its commit history. The GitHub repository serves as supplementary evidence only.

Criterion	What is assessed	Marks
Runnable & Complete	The system starts with `docker compose up` by following the README; the required functionality of the increment is implemented; the technology constraints are respected.	12
Code Quality & Documentation	Code is readable and well structured and matches the documented architecture (services, network communication, events, APIs); README and `/docs` are complete as specified in Section 3.1 / 3.2.	16
Feature Design / Testing	Assessment 1: design of the chosen additional features, covering the six points in Section 3.1. Assessment 2: automated unit and integration tests running in the GitHub Actions CI pipeline, test catalogue.	6
Scrum Process	Scrum artefacts in `/docs` (backlog, sprint log, reviews, retrospectives); regular, meaningful commits from all team members in the submitted commit history.	6
Total		**40**

5.3 Individual Reflective Report Marking (40 marks)

Worth **50% of each assessment** (25% of the final module grade). The report is marked individually based on your personal contribution and reflection.

Criterion	What is assessed	Marks
Contribution & Collaboration	Own contribution is clearly described, distinguished from team work, and supported by code/commit references.	12
Technical Decisions	Decisions are explained with a clear rationale and with alternatives considered.	12
Challenges & Learning	Challenges, solutions, and learning are reflected on critically and realistically.	12
Clarity	Logical structure and clear, correct writing.	4
Total		**40**

If tasks are incomplete by the assessment deadline, submit the report anyway and explain which parts are incomplete, what problems caused the delay, and how you attempted to resolve them.

After each assessment, you will receive feedback based on these marking schemes, which you can incorporate into the next assessment.

6 Submission

6.1 Submission Deadlines

- **Assessment 1:** Upload by **11.11.2026, 12:00** (Moodle) — Presentations: **12.11.2026**
- **Assessment 2:** Upload by **16.12.2026, 12:00** (Moodle) — Presentations: **17.12.2026**

6.2 What to Submit

Each student submits the following to Moodle by the deadline:

A. Repository snapshot: zip including the `.git` folder, without build artefacts
B. Link to the team's GitHub repository, with access granted to both markers (supplementary evidence)
C. Individual reflective report (PDF, 2 pages maximum, plus GenAI declaration appendix)
D. Presentation slides (PDF or PowerPoint)

What must be in the repository:

The repository must contain everything needed to run and understand your system:

- README.md with high-level architecture (Mermaid diagrams recommended), setup instructions, and how to run
- Complete source code with `docker-compose.yml`
- `/docs` folder with detailed documentation (see specific requirements for each assessment)
- Tests and CI configuration (Assessment 2)

Important notes:

- Tag the submitted commit as `assessment-1` or `assessment-2`. The submitted repository snapshot (matching this tag) is the version used for marking; the GitHub repository may change or be deleted later and is used only as supplementary evidence.
- Git commit timestamps count as submission evidence—ensure all team members have regular, meaningful commits throughout the project
- Work continuously via GitHub throughout the project period

6.3 Late Submissions

Standard Lancaster University late submission penalties apply to all uploads (repository snapshot, GitHub link, reflective report, and slides). If exceptional circumstances affect your ability to meet a deadline, follow the Exceptional Circumstances procedure described in the Module Handbook.

6.4 Technical Requirements

- **Runnable code:** Your system must be executable using `docker compose up`. Include all necessary build automation (Maven, Gradle, npm, etc.).
- **No build artefacts:** Never include compilation results (`target/`, `build/`, `node_modules/`, `.venv/`, etc.) in your repository—these are generated automatically.
- **Complete documentation:** Your README.md must contain clear setup and run instructions. Code is only considered runnable if it can be executed following your README.
- **Team integration:** If your component requires other team members' components, include everything in the repository and document individual parts clearly.

For further questions or discussion, please contact the module convenor: m.babazadeh@lancaster.ac.uk